

Project Report: Applied Machine Learning with Python (TECH-27, Summer 2026)

Project Title: From Pixels to Positions: Chess FEN Generation via Machine Learning

Team: Mahidas Prabhanjan (Individual Project)

Date: 09/07/2026

1. Objective

The primary objective of this project is to accurately translate a raw 2D image of a chessboard into its standardized Forsyth-Edwards Notation (FEN) string. FEN is a standard notation for describing a particular board position of a chess game, containing all the necessary information to restart a game from that exact position (Ref: [FEN](#)).

2. Motivation

In standard image classification tasks, achieving a 98% accuracy is generally considered highly successful. However, chess is a domain of absolute precision. If a model predicts 63 out of 64 squares correctly (yielding a 98.4% square-level accuracy), the resulting FEN string represents an invalid or materially altered game state, equating to a 0% exact board accuracy. Therefore, automated FEN generation serves as an extreme stress test for machine learning models, forcing them to achieve near-perfect spatial recognition, feature extraction, and resilience against severe class imbalances.

3. Methodology

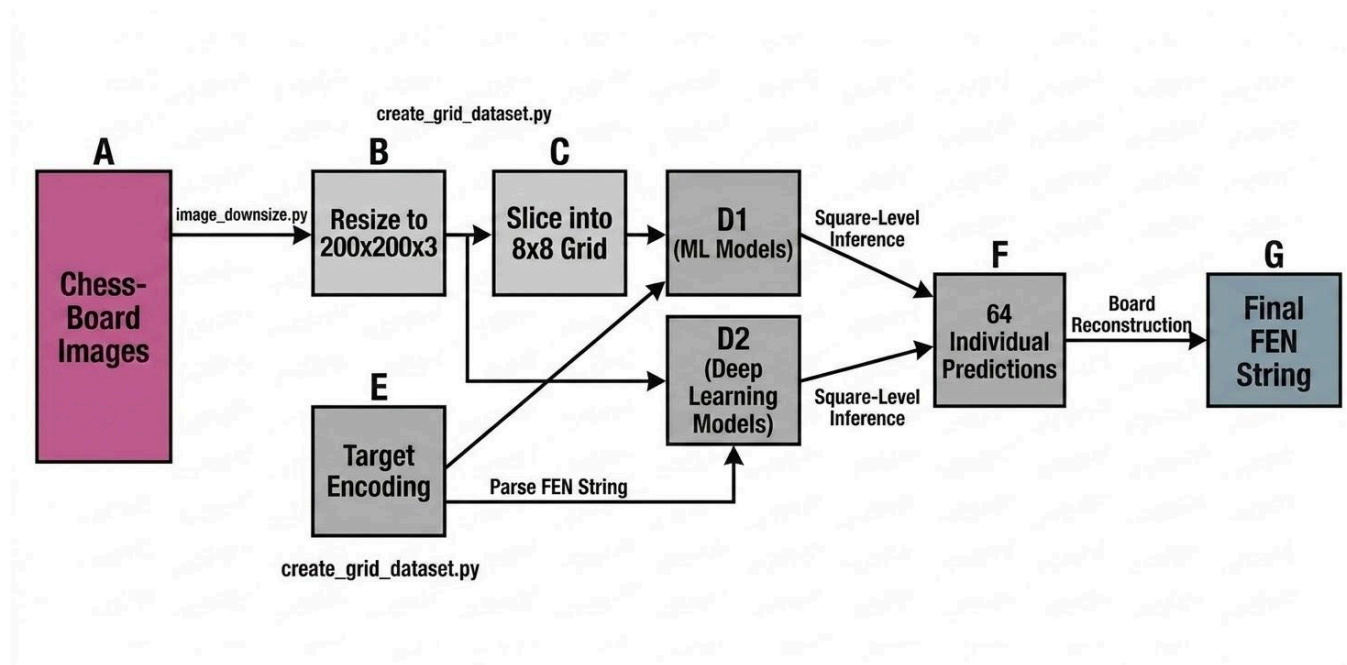
The problem of predicting an entire 64-square board state simultaneously is highly complex. For basic models, the methodology breaks this problem down into a localized, square-by-square pipeline. The details are as follows:

- **Image Preprocessing:** High-resolution raw images are mathematically downsized to a uniform 200x200 pixel RGB format. This is common to all models.
- **Basic Models:**
 - **Grid Slicing:** The 200x200 image is algorithmically cropped into a grid of 64 individual 25x25 pixel squares. This is used for basic models (i.e. non Neural/Deep-Learning Networks).
 - **Target Encoding:** The ground-truth FEN string is parsed and expanded into 64 distinct integer labels representing the 12 piece types and empty squares (totaling 13: {K, Q, R, B, N, P, k, q, r, b, n, p, empty}). This is needed for localized square-by-square prediction in basic models.
 - **Square-Level Inference:** The models evaluate the 64 squares

individually, predicting the class of each.

- **Board Reconstruction:** The 64 individual predictions are sequenced and compressed back into the standard FEN notation.
- **Neural/Deep-Learning Models:**
 - Here, instead of training the models at the localized square-by-square level, the entire 200x200 image is fed.
 - The models output 64 predictions, one for each square, and each square can have one of 13 labels.
 - The 64 outputs can then be either compared against the expected FEN output (during model training & evaluation), or used to construct FEN state of the board output (during inferencing)

6. Block Diagram



7. Tools and Techniques

- **Languages & Environments:** Python 3.11, Google Colab (leveraging Apple

M-series GPU execution via Metal). For Deep-Learning Models, a local Jupyter lab running on Mac M5 Pro Max with 128GB was used.

- **Libraries:** TensorFlow/Keras (for Deep Learning), Scikit-Learn (for traditional ML), OpenCV (for Computer Vision preprocessing), PIL (Image manipulation).
- **Feature Engineering:** For traditional ML models (Logistic Regression, k-NN, SVM, etc.), the 25x25x3 RGB squares were flattened into 1D vectors containing 1,875 features. For Multi Layer Perceptron & Convolutional Neural Networks, the spatial 2D matrix structure was preserved to allow spatial feature extraction via 2D kernels.
- **Machine Learning Models Evaluated (8 Total):** Logistic Regression, k-Nearest Neighbors (k=5), Random Forest, Support Vector Machine (RBF Kernel), Histogram Gradient Boosting, Multi-Layer Perceptron (MLP), Base CNN, and Advanced CNN.

8. Evaluation Metrics

To ensure a robust evaluation of model performance, four specific metrics were tracked:

1. **Square-Level Accuracy:** The percentage of individual pieces correctly identified.
2. **Exact Board (FEN) Accuracy:** The Perfect Match Rate. The percentage of full 64-square boards where every single square was predicted flawlessly.
3. **F1-Score (Per-Class and Weighted):** Crucial for evaluating performance on rare pieces (Kings/Queens) against the dominant "empty" class.
4. **Inference Latency:** The time required in seconds to process and predict the board state, vital for real-time application viability.

9. Success Criteria

The project aims to achieve an **Exact Board FEN Accuracy of > 85%** on unseen test data with the most advanced model (Advanced-CNN),, aligning with the TECH 27 numerical definition requirements. The stretch goal is an exact board accuracy of > 95% with sub-second inference latency.

Although the 85% accuracy is the most important & tangible success criteria, the indirect and/or intangible ones are:

- Successful data-engineering of a Chessboard image for training and evaluation.
- Successful demonstration of using GPUs to accelerate the models training &

evaluation on a large set of data (80K training set & 20K test set) for complex models.

10. Failure Criteria

The project will be considered a failure if the Exact Board FEN Accuracy is **< 50%**, or if a model achieves a high overall square-level accuracy solely by defaulting to predicting the majority class ("empty square").

11. Project Timeline

- **Weeks 1:** Data Engineering (Dataset acquisition, downsizing, grid slicing, balancing scripts).
- **Weeks 2:** Traditional Machine Learning evaluation (LR, k-NN, RF, SVM, Boosting); using evaluation metrics. .
- **Weeks 3-4:** Deep Learning architectures (MLP, CNN & Adv-CNN) and Computer Vision pipelines for real-world photo inference.

12. Implementation Details

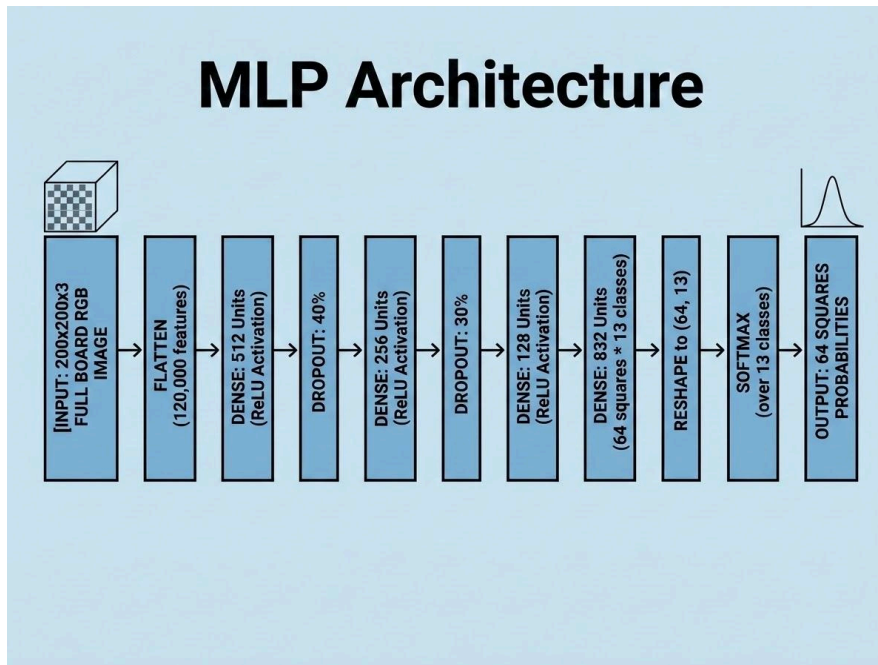
The data preprocessing pipeline was custom-built using four distinct Python modules:

- **image_downsize.py:** Utilized Lanczos resampling to standardize high-resolution inputs to 200x200.
- **create_grid_dataset.py:** Sliced the board into 64 sub-images, parsing the FEN string to assign accurate labels.
- **create_balanced_dataset.py:** A standard chessboard starts with 32 empty squares, creating a massive class imbalance. This script randomly sampled images to ensure an equal distribution of pieces and empty squares in the training data, preventing majority-class bias.
- **filter_unused_boards.py:** To prevent data leakage, this script quarantined 200 full-board images that were never used to generate grid slices, creating an uncontaminated inference vault for the final Exact Board evaluations.
- **tech27_project_chess_basic_models.ipynb:** Implementation and evaluation of all the five basic machine learning models: Logistic-Regression, k-NN (5), Random

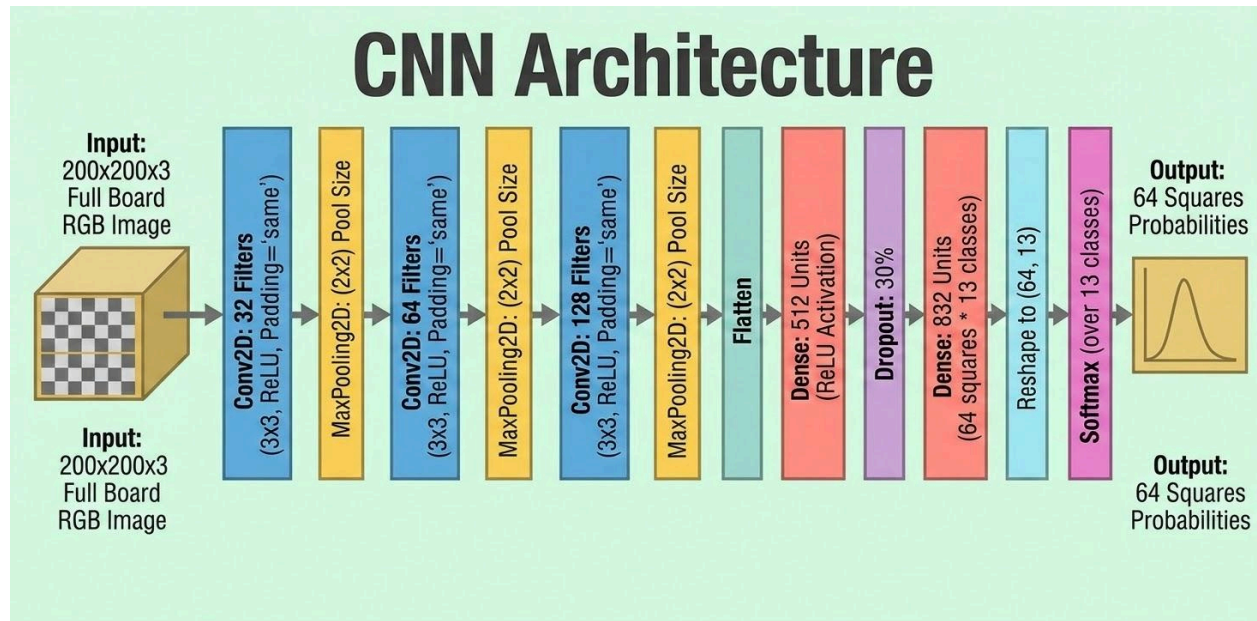
Forest, SVM & Boosting.

- **chess_fen_neural_networks.ipynb**: Neural/Deep-Learning Models starting from Multi-Layer Perceptron to CNN to Advanced-CNN.
- **Misc**: Python11 virtual ENV with all dependencies

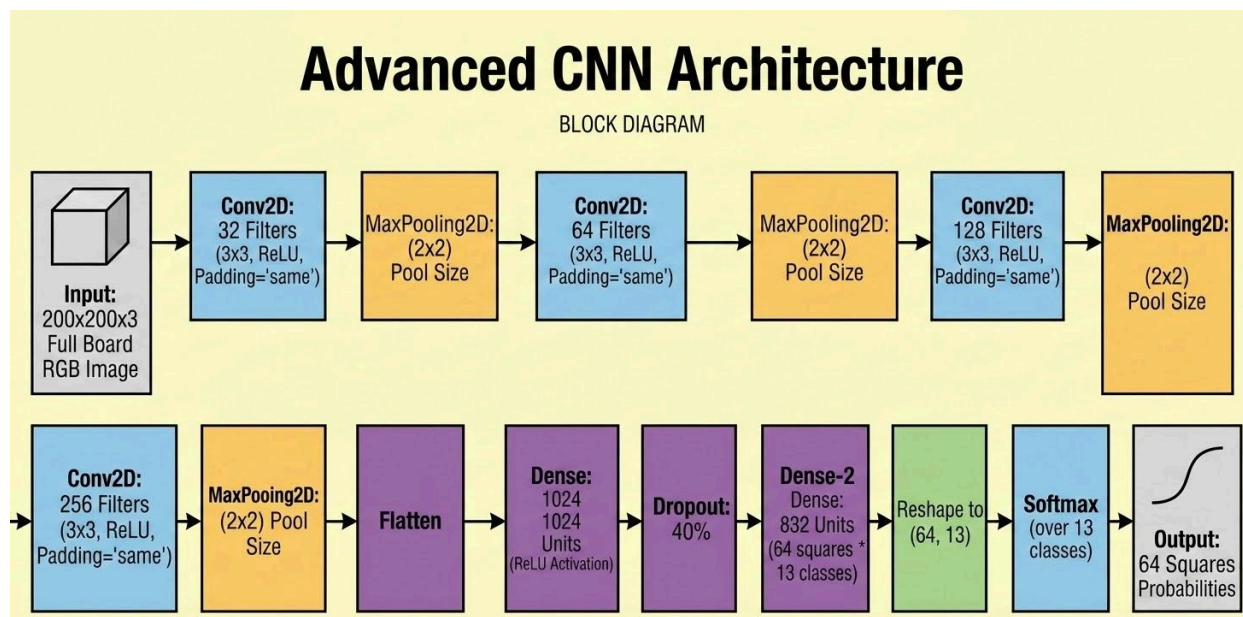
12.1 Multi-Layer Perceptron



12.2 Convolutional Neural Network



12.3 Advanced Convolutional Neural Network



13. Experimentation and Results

- **Traditional ML Models:** Logistic Regression, Random Forest, SVM, and Histogram Gradient Boosting achieved near 100% accuracy on the pristine, synthetic grid datasets. However, **k-NN and SVM suffered severe inference latency penalties** (taking 553 and 689 seconds respectively to evaluate the test set), disqualifying them for real-time applications.
- **Deep Learning - MLP:** The Multi-Layer Perceptron failed the project criteria, stalling at a 63.64% test accuracy. The confusion matrix revealed the MLP failed to learn spatial structures and achieved its score almost entirely by guessing the "empty" class.
- **Deep Learning - CNN:** Moving to a CNN allowed 2D kernels to extract piece shapes. The base CNN improved accuracy to ~99%, but struggled slightly with structurally similar pieces (e.g., mixing up Kings and Queens).
- **Advanced CNN:** By expanding the architecture to 4 Convolutional blocks (up to 256 filters) and 1,024 Dense neurons, the Advanced CNN reached a **99.99% Exact Board Accuracy**, successfully capturing complex piece geometries.

14. Test Cases

The models were evaluated against three distinct datasets to test generalization:

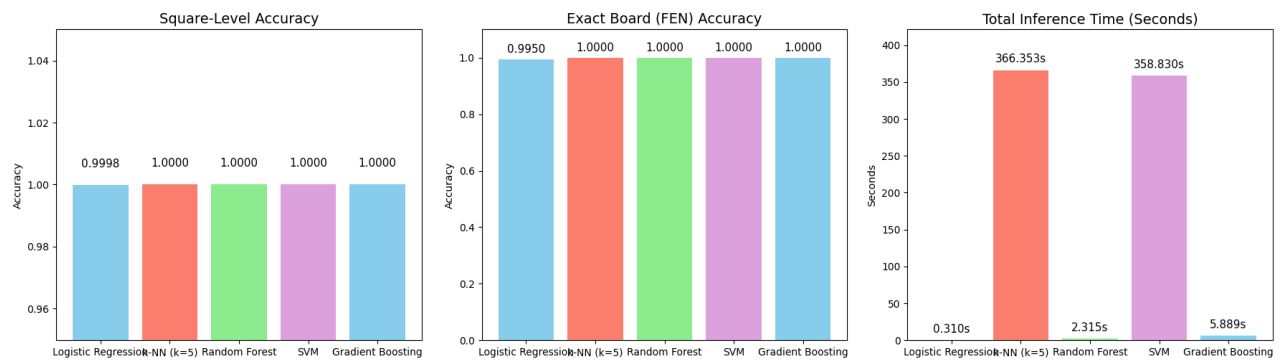
1. **Grid Test Set:** 20,000 isolated 25x25 piece squares. The grid came out of the same original Kaggle Dataset. This was obtained by slicing the original *test* part of the original Kaggle dataset, but was rebalanced to remove the heavy skewness towards 'empty' squares.
2. **Uncontaminated Full Boards:** 200 full 200x200 synthetic boards isolated from the training generation pipeline. This is part of the test/ subset of the original Kaggle Dataset, but this was not used in the square-by-square training.
3. For the Deep Learning Models, the test subset of the original Kaggle Dataset,

totaling 20K images, was fully utilized to evaluate the models.

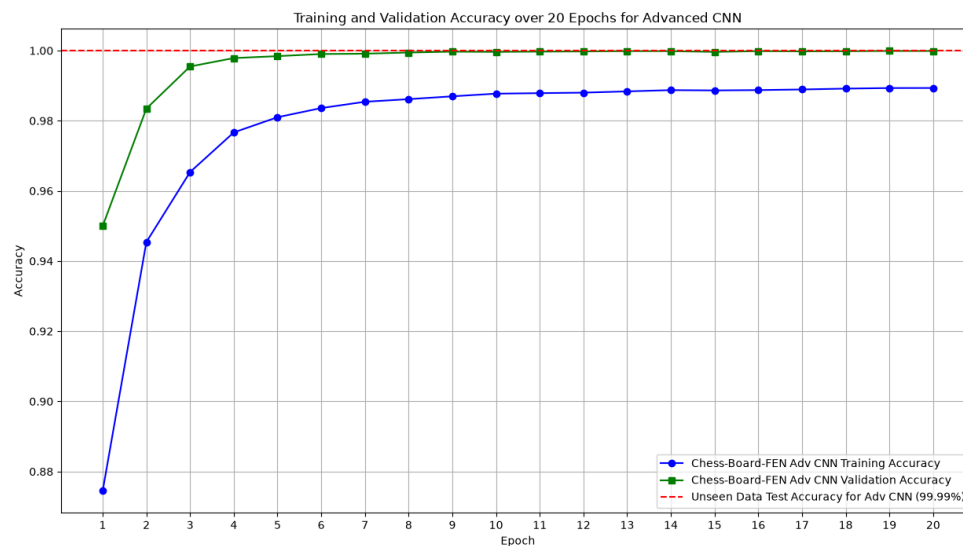
4. **Real-World Images:** Digital screenshots and angled photos of chessboards sourced from the web.

15. Graphs, Plots, and Performance Figures

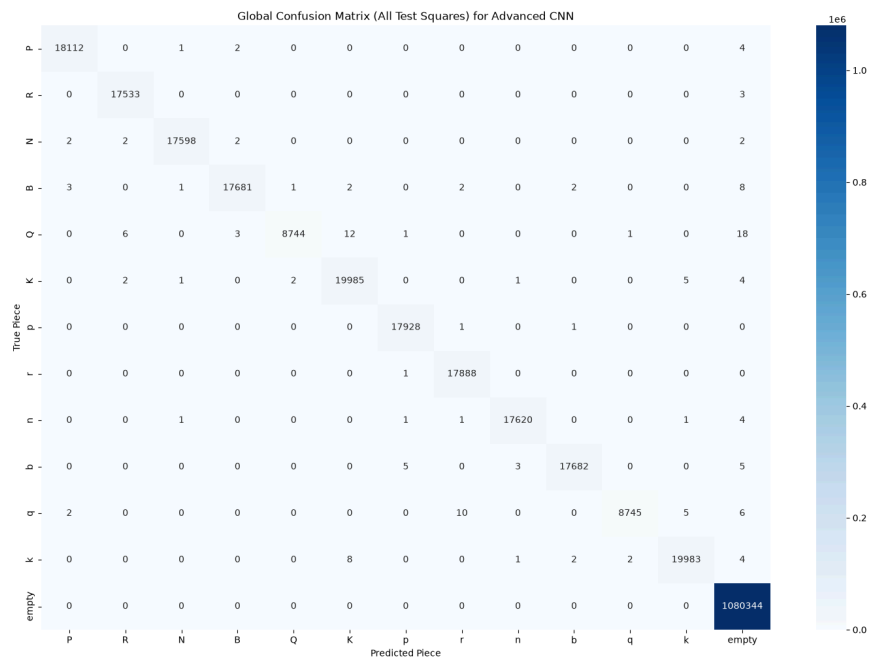
1: Bar Chart comparing Test Accuracies of Traditional ML models



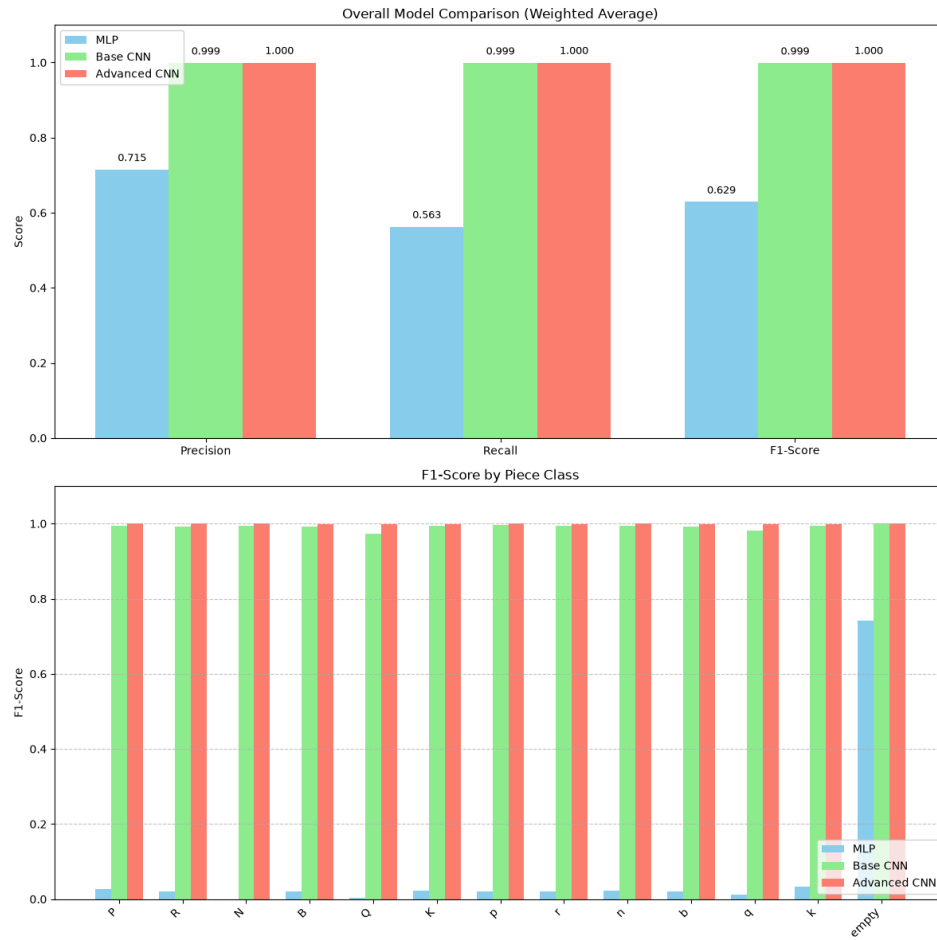
2: Graph of Adv-CNN Training/Validation Accuracy over 20 Epochs



3: Heatmap of Global Confusion Matrix for Advanced CNN



4: Grouped Bar Chart of Precision/Recall/F1-Score across MLP, Base CNN, and Adv-CNN



5: 3-Panel Visual Output of Real-World Inferencing of Few Images from the Internet

[Image 1/6]: image_1.jpeg

image_1.jpeg (No Crop)



CNN PREDICTION:

.	.	.	.	.	.	.	.	.	.
N	p	.	.	.	.	.	b	p	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	P	.	.	.	.
.	.	.	.	.	.	P	N	.	.
.	.	.	p	k	.	B	.	.	.
P	.	.	.	.	.	.	P	.	.
.	.	.	.	.	.	K	.	.	.

=====

[Image 2/6]: image_2.jpeg

Original: image_2.jpeg



Cropped (4%)



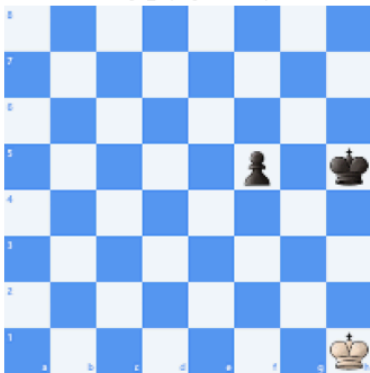
CNN PREDICTION:

.	r	.	.	.	.	.	.	.	.
.	p	p	b	.	p	b	.	.	.
.	.	P	.	n	n	p	.	.	.
.	.	.	.	p	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	N	.	.	.	P	.	.	.
N	N	.	.	.	.	P	.	.	.
.	.	K	.	.	.	B	.	.	.

=====

[Image 3/6]: image_3.png

image_3.png (No Crop)



CNN PREDICTION:

.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	p	.	k	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	k	.

=====

[Image 4/6]: image_4.png



CNN PREDICTION:

	.	.	.	.	.	.	.	.	.	
	.	.	.	P	K	.	.	.	.	
	.	.	.	.	.	N	k	.	.	
	.	R	.	.	.	.	.	.	.	
	.	.	.	P	.	p	.	.	.	
	.	.	.	.	.	b	.	.	.	
	.	.	.	.	.	.	.	.	.	
	r	.	.	.	.	.	.	.	.	

=====

[Image 5/6]: image_5.jpg

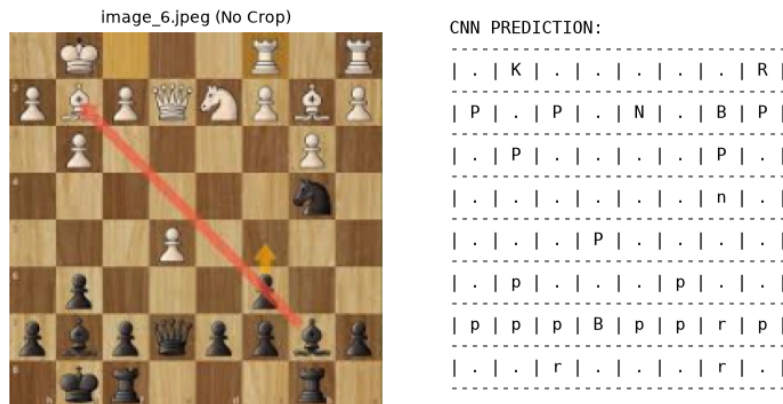


CNN PREDICTION:

	.	.	.	.	.	.	.	.	r	
	.	.	p	q	k	.	.	.	.	
	P	.	.	.	.	p	.	p		
	.	.	p	.	p	.	.	.		
	.	.	r	n	.	.	.	.		
	.	.	.	.	Q	N	.	.		
	.	.	.	.	.	P	P	P		
	R	.	.	R	.	.	k	.		

=====

[Image 6/6] Processing: image_6.jpeg



16. Optimization Techniques Used

To maximize the Deep Learning models without overfitting, several optimization callbacks were utilized:

- **Aggressive Dropouts (0.3/0.4):** Applied to the Dense layers of the Advanced CNN to force generalization.
- **ReduceLROnPlateau:** Dynamically halved the learning rate when validation loss stagnated for 4 epochs, preventing the optimizer from overshooting the global minimum.
- **EarlyStopping:** Monitored val_loss and halted training completely if no improvements were seen after 5 epochs, restoring the best weights.
- **Actual Observation:** Observed the Deep Learnings models encounter EarlyStopping due to the rules defined called out above.

17. Standards Considered, Constraints, and Limitations

- **Constraint:** The system requires all 64 squares to be visible. It cannot handle heavily occluded (i.e. blocked/obstructed) boards, such as a hand physically moving a piece across the frame.
- **Limitation:** The synthetic dataset used for primary training lacks 3D shadows,

dynamic lighting, and variable camera angles. Generalization to physical wooden chessboards is currently limited without extensive data augmentation. This resulted in the basic models achieving near 100% accuracy.

18. Challenges Faced and How You Addressed Them

The primary challenges occurred during the real-world computer vision inference phase:

- **Contour Detection Failure:** When using OpenCV to find the chessboard in a real photo, the findContours algorithm frequently latched onto a single internal square rather than the full board perimeter.
 - *Solution:* Implemented a total image area threshold, forcing the algorithm to reject any detected polygon that constituted less than 40% of the image's total footprint.
- **Grid Misalignment:** Screenshots containing coordinate borders (A-H, 1-8) physically pushed the chessboard squares inward, misaligning them with the CNN's mathematical slicing logic.
 - *Solution:* Engineered a dynamic fallback cropper that calculates image dimensions and shaves exactly 4% off the margins of digital screenshots before passing them to the CNN. But even this did not yield satisfactory results as the border areas in real-world images are not fixed and their effective removal is a challenge in itself.

19. Conclusion and Future Work

The project successfully demonstrated that an Advanced Convolutional Neural Network, when paired with a robust OpenCV preprocessing pipeline, can effectively translate raw pixels into perfect FEN representations. Achieving a 99.99% exact board accuracy on unseen data vastly surpassed the 80% success criteria threshold.

Future Work

- The natural next step is deploying extreme Data Augmentation (e.g., random perspective warping, brightness shifts, and synthetic shadows) during training to adapt the model for 3D physical boards.
- Additionally, converting the heavy keras model to TensorFlow Lite would allow for real-time inference deployment directly on edge mobile devices.
- Memory and Runtime Optimizations.
- The square level accuracy of real-world unseen images has still quite a bit of scope for improvement.
- Effectively cropping out the noise in the images prior to inferencing, or to effectively train the model with these noise present in the training data.
- Problem Inversion: Generate Full Chessboard Image from a FEN String!

20. Ethics and Bias

- **Training Bias:** Because the model was trained exclusively on 2D digital board representations, it carries a heavy bias toward online chess assets. This restricts equitable application, as the model will perform poorly on standard physical boards used in over-the-board tournaments.
- **Ethical Considerations:** Automated FEN extraction from screen captures is a dual-use technology. While it is beneficial for quickly digitizing historic games from textbooks, it can easily be hooked into chess engines (such as Stockfish) to facilitate malicious cheating bots in online chess environments. Care must be taken to ensure the technology is used for analysis, not illicit competitive advantage.